

## Assignment 3

Name: Esraa Ashraf

ID: 224866

### Part 1: Advanced Theoretical Assignment

#### Q1. Syntax Troubleshooting and Error Identification

The first port declaration is missing a direction. It should be declared as an input, output, or inout.

The module uses ANSI-style port declarations, so every listed port should have its direction specified.

The memory declaration `grid_mem [0:3][0:1]` is a two-dimensional memory of 8-bit elements and is valid.

`grid_mem[0] = 8'hFF` is invalid because `grid_mem[0]` still represents a complete second dimension. A complete memory element such as `grid_mem[0][0]` must be selected.

`grid_mem[1][2:3]` is invalid because the second memory dimension only contains indices 0 and 1, and a memory dimension cannot be used as a packed bit-select.

The part-select `full_word[start_idx + 3 : start_idx]` uses variable boundaries. In Verilog-2001, a variable part-select should use `+`: or `-`:

`full_word` has no driver, so its value will be unknown during simulation. This is not a syntax error, but it makes the result of the part-select unknown.

`data_bus` is declared as a 1-bit inout wire. This is legal, although it is unused in the original code.

A corrected version is provided in `q1_fixed_memory_ctrl.v`.

#### Q2. Tri-State Logic and Bus Contention

Standard binary states 0 and 1 are not sufficient for a shared bus because more than one device may be connected to the same physical wires. When a device is not transmitting, it must disconnect electrically from the bus. The high-impedance state (Z) provides this disconnected state. Without it, two active drivers could drive opposite values at the same time, causing bus contention.

Tri-state buffer:

```
module tri_state_buffer(  
    input wire [7:0] tx_data,  
    input wire tx_en,  
    inout wire [7:0] shared_bus  
);
```

```
assign shared_bus = tx_en ? tx_data : 8'bz;

endmodule
```

### Q3. Data Type Semantics

wire is a net type. It represents a connection and is normally driven by continuous assignments, module outputs, or primitive outputs. A wire cannot be assigned inside a procedural always block.

reg is a variable type used for values assigned inside procedural blocks such as always or initial. A reg can also be driven by a procedural assignment. The name reg does not necessarily mean a physical flip-flop; synthesis determines the hardware from how it is used.

Both wire and reg start with an unknown value (X) in simulation unless they receive an initial/continuous/procedural assignment or another driver.

## Part 2: Practical Lab Exercise

### Task 1: Data Router Design

The module data\_router uses a continuous assignment for byte\_low and parity\_bit, and an always block for byte\_high as required.

```
module data_router(
input wire [31:0] data_in,
output reg [7:0] byte_high,
output wire [7:0] byte_low,
output wire parity_bit
);

assign byte_low = data_in[7:0];
assign parity_bit = data_in[14];

always @(*) begin
byte_high = data_in[31:24];
end

endmodule
```

### Task 2: Register File Simulation

The register file contains eight 16-bit locations. Writing occurs synchronously on the positive clock edge, while reading is asynchronous.

```
module reg_file(
input wire clk,
input wire [2:0] read_addr,
```

```

input wire [2:0] write_addr,
input wire [15:0] write_data,
input wire we,
output reg [15:0] read_data,
output wire [7:0] read_data_top_byte
);

reg [15:0] register_file [0:7];

always @(posedge clk) begin
if (we)
register_file[write_addr] <= write_data;
end

always @(*) begin
read_data = register_file[read_addr];
end

assign read_data_top_byte = read_data[15:8];

endmodule

```

### Task 3: Configurable 2D Grid Memory Interface

The grid\_mem\_router module uses a 4-by-2 memory of 8-bit elements, a tri-state bus, an asynchronous active-high reset, and variable part-select operators for assembling the 32-bit processing word.

```

module grid_mem_router #(
parameter WORD_WIDTH = 32)
(input wire clk,
input wire Rst,
input wire oe,
input wire endian_swap,
input wire [1:0] row_addr,
input wire col_addr,
output reg [31:0] processing_word,
inout wire [7:0] bus_data);

reg [7:0] fabric_mem [0:3][0:1];
integer i;

assign bus_data = oe ? fabric_mem[row_addr][col_addr] : 8'bz;

always @(posedge clk or posedge Rst) begin

```

```

if (Rst) begin
processing_word <= 32'b0;
end

else begin

if (!endian_swap) begin
for (i = 0; i < 4; i = i + 1)
processing_word[i*8 +: 8] <= fabric_mem[i][0];
end

else begin
for (i = 0; i < 4; i = i + 1)
processing_word[(i+1)*8-1 -: 8] <= fabric_mem[3-i][0];
end

end

end

endmodule

```

## Files Included

q1\_fixed\_memory\_ctrl.v  
tri\_state\_buffer.v  
data\_router.v  
reg\_file.v  
grid\_mem\_router.v  
tb.v